

BO_ML4F — Implementation Notes

Bayesian Optimisation with Gaussian Processes,
Rank-1 Cholesky Updates, Multi-Restart HPO,
Matérn 5/2 Kernel, and Output Normalisation

Y. Lecomte

M. Mendez

July 2026

Contents

1	Overview	2
2	Kernel Functions	2
2.1	Squared-Exponential (RBF) Kernel	2
2.2	Matérn 5/2 Kernel	3
2.3	Kernel Dispatcher	3
3	Gaussian Process Regression	4
3.1	GP Fitting via Cholesky Factorisation	4
3.2	GP Prediction	5
3.3	Output Normalisation	5
4	Hyperparameter Optimisation (HPO)	6
4.1	Log Marginal Likelihood	6
4.2	Multi-Restart Strategy with Warm Starting	7
5	Acquisition Functions	8
5.1	Expected Improvement (EI)	8
5.2	Probability of Improvement (PI)	8
5.3	Lower Confidence Bound (LCB/UCB)	8
6	Acquisition Optimisation	8
7	Rank-1 Cholesky Update	9
7.1	Motivation and Complexity	9
7.2	Block Cholesky Extension	9
7.3	Alpha Update	9
7.4	Integration into the BO Loop	10
8	The BO Loop	11
9	Configuration Reference	11
10	Test Cases and Examples	12
10.1	1D_Test_CASE.py — 1D example with BO_ML4F.py	12
10.2	3D_Test_CASE.py — 3D example with BO_ML4F.py	13
10.3	Development-time references (not included in this repository)	13

1 Overview

This document describes the mathematics and Python implementation of `B0_ML4F.py`, a self-contained Bayesian Optimisation (BO) library developed for the RSPA paper on the Burger equation control case. A secondary helper module, `B0_func_YL.py`, provides a lower-level implementation used in earlier prototypes.

The design goals are:

1. **Correctness** — exact GP posterior, proper acquisition functions for both minimisation and maximisation.
2. **Efficiency** — rank-1 Cholesky updates (Section 7) reduce the per-iteration cost from $\mathcal{O}(n^3)$ to $\mathcal{O}(n^2)$ whenever the hyperparameters are held fixed.
3. **Robustness** — multi-restart hyperparameter optimisation with warm starting (Section 4) and output normalisation (Section 3.3) for well-conditioned GP fitting.
4. **Flexibility** — two interchangeable kernels (squared-exponential and Matérn 5/2, Section 2) selectable via a single configuration field.
5. **Modularity** — all settings are grouped into typed dataclasses (`GPConfig`, `AcqConfig`, `OptimConfig`, `BOConfig`, `SaveConfig`).

Remark 1. All line numbers cited in this document refer to `B0_ML4F.py` as it stands after the July 2026 revision that introduced the rank-1 update, multi-restart HPO, Matérn 5/2 kernel, and output normalisation.

2 Kernel Functions

The covariance function $k(\mathbf{x}, \mathbf{x}'; \boldsymbol{\theta})$ encodes prior beliefs about the smoothness of the objective. Two kernels are implemented, both parameterised by the same triple $\boldsymbol{\theta} = (\ell, \sigma_f, \sigma_y)$ (length-scale, signal amplitude, noise standard deviation).

2.1 Squared-Exponential (RBF) Kernel

The squared-exponential kernel (also called the radial basis function, RBF, or Gaussian kernel) is

$$k_{\text{SE}}(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right) = \sigma_f^2 \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2), \quad \gamma = \frac{1}{2\ell^2}. \quad (1)$$

This kernel is infinitely differentiable, implying infinitely smooth sample paths. It is a good default for well-behaved analytic functions but can over-smooth physical simulation outputs.

Code. `rbf_kernel_amp()` (lines 333–341) delegates pairwise distance computation to `rbf_kernel_()` in `B0_func_YL.py` (lines 24–26):

Listing 1: `rbf_kernel_amp()` in `B0_ML4F.py`

```

333 def rbf_kernel_amp(X1: Array, X2: Array, l_c: float, sigma_f: float) ->
    Array:
334     gamma = 0.5 / (l_c**2)
335     return (sigma_f**2) * rbf_kernel_(X1, X2, gamma=gamma)

```

Listing 2: `rbf_kernel_()` in `BO_func_YL.py`

```

24 def rbf_kernel_(X1, X2, gamma):
25     sqdist = np.sum((X1[:, None, :] - X2[None, :, :])**2, axis=2)
26     return np.exp(- gamma * sqdist)

```

The broadcasting trick `X1[:,None,:] - X2[None,[:, :]]` computes all $n_1 \times n_2$ pairwise differences in one vectorised operation, avoiding any Python loop.

2.2 Matérn 5/2 Kernel

The Matérn kernel with smoothness parameter $\nu = 5/2$ is

$$k_{5/2}(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \left(1 + \frac{\sqrt{5}r}{\ell} + \frac{5r^2}{3\ell^2} \right) \exp\left(-\frac{\sqrt{5}r}{\ell}\right), \quad r = \|\mathbf{x} - \mathbf{x}'\|_2, \quad (2)$$

where ℓ and σ_f play the same roles as in the RBF kernel. Sample paths are exactly *twice* differentiable — a more realistic prior for physical simulations whose outputs are not infinitely smooth.

For both kernels, evaluating at $r = 0$ gives $k(\mathbf{x}, \mathbf{x}) = \sigma_f^2$ (the bracketed factor in Eq. (2) equals unity when $r = 0$). This identity is exploited in Section 3.2 to avoid forming the full $m \times m$ diagonal of $\mathbf{K}_{**}$.

Comparison with RBF. For a fixed length-scale ℓ , the Matérn 5/2 kernel decays more slowly with distance than the RBF kernel; equivalently, it places more prior mass on non-smooth functions. In practice this makes HPO more robust when the objective has sharp features, and it is the default choice in production libraries such as `scikit-optimize`.

Code. `matern52_kernel_amp()` (lines 343–353):

Listing 3: `matern52_kernel_amp()` in `BO_ML4F.py`

```

343 def matern52_kernel_amp(X1: Array, X2: Array, l_c: float, sigma_f: float
344     ) -> Array:
345     sqdist = np.sum((X1[:, None, :] - X2[None, :, :])**2, axis=2)
346     r      = np.sqrt(np.maximum(sqdist, 0.0))    # numerical safety for r
347     ~0
348     a      = np.sqrt(5.0) * r / l_c
349     return (sigma_f**2) * (1.0 + a + a**2 / 3.0) * np.exp(-a)

```

The `np.maximum(sqdist, 0.0)` guard prevents negative arguments to `sqrt` that can arise from floating-point cancellation when two points are nearly coincident.

2.3 Kernel Dispatcher

All internal functions — `gp_fit()`, `gp_predict()`, `negative_log_marginal_likelihood()`, `update_gp_rank1()` — call a single dispatcher `kernel_amp()` (lines 356–370) that routes to the appropriate implementation based on a string key:

Listing 4: `kernel_amp()` dispatcher in `BO_ML4F.py`

```

356 def kernel_amp(X1: Array, X2: Array, l_c: float, sigma_f: float,
357     kind: str = "rbf") -> Array:
358     if kind == "rbf":
359         return rbf_kernel_amp(X1, X2, l_c, sigma_f)
360     elif kind == "matern52":
361         return matern52_kernel_amp(X1, X2, l_c, sigma_f)

```

```

362     else:
363         raise ValueError(f"Unknown kernel '{kind}'. Choose 'rbf' or 'matern52'.")

```

The kernel choice is stored in `GPConfig.kernel` (line 71) and propagated to `GPMoel.kernel` (line 136) via `build_gp_model()` (line 704). Switching kernels requires only `GPConfig(kernel="matern52")` — no other changes are needed.

3 Gaussian Process Regression

A Gaussian process is a distribution over functions:

$$f(\mathbf{x}) \sim \mathcal{GP}(0, k(\mathbf{x}, \mathbf{x}'; \boldsymbol{\theta})).$$

Given n noisy observations $\mathcal{D}_n = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ with $y_i = f(\mathbf{x}_i) + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0, \sigma_y^2)$, the posterior at a test point $\mathbf{x}_*$ is

$$\mu_* = \mathbf{k}_*^\top (\mathbf{K}_n + \sigma_y^2 \mathbf{I})^{-1} \mathbf{y}_n, \quad (3)$$

$$\sigma_*^2 = k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}_*^\top (\mathbf{K}_n + \sigma_y^2 \mathbf{I})^{-1} \mathbf{k}_*, \quad (4)$$

where $[\mathbf{K}_n]_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ and $\mathbf{k}_* = [k(\mathbf{x}_i, \mathbf{x}_*)]_{i=1}^n$.

3.1 GP Fitting via Cholesky Factorisation

Direct inversion of $\mathbf{K}_y = \mathbf{K}_n + \sigma_y^2 \mathbf{I}$ is $\mathcal{O}(n^3)$ and numerically fragile. Instead we compute the Cholesky factorisation

$$\mathbf{K}_y = \mathbf{L}\mathbf{L}^\top, \quad \mathbf{L} \in \mathbb{R}^{n \times n} \text{ lower triangular}, \quad (5)$$

and solve for the weight vector $\boldsymbol{\alpha}$ in two triangular sweeps:

$$\boldsymbol{\alpha} = \mathbf{K}_y^{-1} \mathbf{y}_n = \mathbf{L}^{-\top} \underbrace{\mathbf{L}^{-1} \mathbf{y}_n}_{\mathbf{v}}. \quad (6)$$

The posterior mean then reduces to the dot product $\mu_* = \mathbf{k}_*^\top \boldsymbol{\alpha}$, which reuses the precomputed $\boldsymbol{\alpha}$ for all test points.

Code. `gp_fit()` (lines 373–387) assembles $\mathbf{K}_y$ using the kernel dispatcher of Section 2.3, calls `scipy.linalg.cholesky`, and solves for $\boldsymbol{\alpha}$ via `cho_solve`:

Listing 5: `gp_fit()` in `BO_ML4F.py`

```

373 def gp_fit(Xs, ys, l_c=0.3, sigma_f=1.0, sigma_y=0.1, jitter=1e-10,
374           kernel="rbf"):
375     Xs = np.asarray(Xs, dtype=float)
376     ys = np.asarray(ys, dtype=float).reshape(-1)
377
378     Kss = kernel_amp(Xs, Xs, l_c=l_c, sigma_f=sigma_f, kind=kernel)
379     n = Xs.shape[0]
380     Ky = Kss + (sigma_y**2 + jitter) * np.eye(n)
381
382     L = cholesky(Ky, lower=True)
383     alpha = cho_solve((L, True), ys)
384     return alpha, L

```

The `jitter` (default 10^{-10} , line 49) is added to the diagonal to ensure strict positive definiteness despite rounding errors.

3.2 GP Prediction

Given (α, L) , prediction at m test points $\mathbf{X}_* \in \mathbb{R}^{m \times d}$ costs only $\mathcal{O}(mn)$:

$$\mu_* = \mathbf{K}_{*n} \alpha, \quad (7)$$

$$\sigma_*^2 = \text{diag}(\mathbf{K}_{**}) - \text{diag}(\mathbf{K}_{*n} \mathbf{V}), \quad \mathbf{V} = \mathbf{K}_y^{-1} \mathbf{K}_{n*}. \quad (8)$$

Since $k(\mathbf{x}, \mathbf{x}) = \sigma_f^2$ for both the RBF and Matérn 5/2 kernels at $r = 0$, we have $\text{diag}(\mathbf{K}_{**}) = \sigma_f^2 \mathbf{1}_m$, computed cheaply as a scalar broadcast — no full $m \times m$ matrix is needed.

Code. `gp_predict()` (lines 390–413):

Listing 6: `gp_predict()` in `B0_ML4F.py`

```

390 def gp_predict(X, Xs, alpha, L, l_c=0.3, sigma_f=1.0, return_cov=False,
391               kernel="rbf"):
392     Ks = kernel_amp(X, Xs, l_c=l_c, sigma_f=sigma_f, kind=kernel)
393     mu = Ks @ alpha
394
395     V = cho_solve((L, True), Ks.T)
396
397     if return_cov:
398         Kxx = kernel_amp(X, X, l_c=l_c, sigma_f=sigma_f, kind=kernel)
399         cov = Kxx - Ks @ V
400         return mu, cov
401     else:
402         # k(x, x) = sigma_f^2 for both RBF and Matern52
403         Kxx_diag = (sigma_f**2) * np.ones(X.shape[0])
404         var = Kxx_diag - np.sum(Ks * V.T, axis=1)
405         return mu, var

```

When output normalisation is active, μ_* and σ_*^2 are returned in the normalised space; denormalisation is the responsibility of the caller (see Section 3.3).

3.3 Output Normalisation

When the objective values y have large absolute magnitude or units far from unity, the GP hyperparameters σ_f and σ_y must also be large to explain the data, which can make the NLL surface ill-conditioned and HPO slow to converge.

The remedy is to work in a normalised space. Before fitting, the observations are centred and scaled:

$$\tilde{y}_i = \frac{y_i - \bar{y}}{s_y}, \quad \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i, \quad s_y = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2}. \quad (9)$$

The GP is fitted on $\tilde{\mathbf{y}}$; the resulting posterior mean $\tilde{\mu}_*$ and variance $\tilde{\sigma}_*^2$ are denormalised at call sites:

$$\mu_* = \tilde{\mu}_* \cdot s_y + \bar{y}, \quad \sigma_*^2 = \tilde{\sigma}_*^2 \cdot s_y^2. \quad (10)$$

This keeps $\sigma_f \sim 1$ and $\sigma_y \sim 1$ regardless of the objective scale, and all acquisition functions (EI, PI, LCB) operate in the original scale after denormalisation.

Remark 2. The normalisation statistics $(\bar{y}, s_y)$ are computed once per full refit and stored in `gp.y_mean` and `gp.y_std` (lines 139–140 of `B0_ML4F.py`). Rank-1 updates reuse the existing statistics — they do *not* recompute $\bar{y}$ and s_y , because updating those would invalidate the Cholesky factor built under the previous normalisation.

Code. The normalisation block is inside `fit_gp()` (lines 715–760), applied before HPO and before the Cholesky call:

Listing 7: Output normalisation in `fit_gp()`

```

726     if gp_cfg.normalize_y:
727         y_mean = float(np.mean(y_raw))
728         y_std = float(np.std(y_raw))
729         if y_std < 1e-12:
730             y_std = 1.0          # guard against constant outputs
731         gp.y_mean = y_mean
732         gp.y_std = y_std
733         gp.ys = (y_raw - y_mean) / y_std
734     else:
735         gp.y_mean = 0.0
736         gp.y_std = 1.0
737         gp.ys = y_raw

```

Denormalisation [Eq. (10)] is applied in `make_acquisition()` immediately after the `gp_predict()` call:

Listing 8: Denormalisation in `make_acquisition()`

```

784     mu_norm, var_norm = gp_predict(
785         Xcand, gp.Xs, gp.alpha, gp.L,
786         l_c=gp.l_c, sigma_f=gp.sigma_f,
787         return_cov=False, kernel=gp.kernel,
788     )
789     # Eq. (4): denormalise to original y scale
790     mu = mu_norm * gp.y_std + gp.y_mean
791     var = var_norm * gp.y_std**2

```

The same denormalisation is applied in `plt_state()` (line 957) so that plotted GP posteriors are always in physical units.

4 Hyperparameter Optimisation (HPO)

4.1 Log Marginal Likelihood

The hyperparameters $\boldsymbol{\theta} = (\ell, \sigma_f, \sigma_y)$ are tuned by maximising the log marginal likelihood (equivalently, minimising the negative log marginal likelihood, NLL):

$$\log p(\tilde{\mathbf{y}} \mid \mathbf{X}, \boldsymbol{\theta}) = -\frac{1}{2} \tilde{\mathbf{y}}^\top \mathbf{K}_y^{-1} \tilde{\mathbf{y}} - \sum_{i=1}^n \log L_{ii} - \frac{n}{2} \log(2\pi), \quad (11)$$

where $\tilde{\mathbf{y}}$ denotes the (possibly normalised) observations stored in `gp.ys`. Because all three hyperparameters are strictly positive, optimisation is performed in log-space $\tilde{\boldsymbol{\theta}} = \log \boldsymbol{\theta}$, so L-BFGS-B operates over $\mathbb{R}^3$ with optional box constraints (`GPConfig.theta.bounds_log`, line 58).

Code. `negative_log_marginal_likelihood()` (lines 568–602) evaluates Eq. (11). A `kernel` argument ensures the NLL is computed with the correct covariance function:

Listing 9: NLL in `B0_ML4F.py`

```

568 def negative_log_marginal_likelihood(
569     theta_log, X_train, y_train,
570     jitter=1e-10, MLE_hist=None, param_hist=None,
571     kernel="rbf",

```

```

572 ):
573     l_c, sigma_f, sigma_y = np.exp(theta_log)
574     K = kernel_amp(X_train, X_train, l_c=l_c, sigma_f=sigma_f, kind=
        kernel)
575     Ky = K + (sigma_y**2 + jitter) * np.eye(X_train.shape[0])
576     L = cholesky(Ky, lower=True)
577     alpha = cho_solve((L, True), y_train)
578     ll = -0.5*(y_train @ alpha) - np.sum(np.log(np.diag(L))) \
579         - 0.5*len(y_train)*np.log(2.0*np.pi)
580     return -float(ll)

```

4.2 Multi-Restart Strategy with Warm Starting

The NLL surface can have multiple local optima, especially when data are sparse. `optimize_gp_hyperparams()` (lines 605–697) addresses this with a three-tier starting-point strategy:

1. **Warm start** (line 640): the solution $\tilde{\theta}^*$ from the *previous* BO iteration is reused. Adding one point rarely shifts the NLL optimum much, so this typically converges in very few L-BFGS-B steps.
2. **User or default start** (lines 644–647): either `gp_cfg.theta0_log` or the current hyperparameters in log-space.
3. **Random restarts** (lines 650–652): `n_hpo_restarts` (default 3, line 65) vectors drawn from $\mathcal{N}(\mathbf{0}, \mathbf{I})$ provide global coverage to escape local optima.

The restart with the lowest NLL is kept (lines 657–670) and its solution stored in `gp.theta_log` (line 695) to seed the next warm start.

Listing 10: Multi-restart loop in `optimize_gp_hyperparams()`

```

637 x0_list: List[Array] = []
638
639 if gp.theta_log is not None:           # (1) warm start
640     x0_list.append(gp.theta_log.copy())
641 if gp_cfg.theta0_log is not None:      # (2) user / default
642     x0_list.append(np.asarray(gp_cfg.theta0_log, dtype=float).
        reshape(-1))
643 else:
644     x0_list.append(np.log([gp.l_c, gp.sigma_f, gp.sigma_y]))
645 rng_hpo = np.random.default_rng()     # (3) random restarts
646 for _ in range(max(0, gp_cfg.n_hpo_restarts)):
647     x0_list.append(rng_hpo.standard_normal(3))
648
649 best_nll, best_theta = np.inf, None
650 for x0 in x0_list:
651     res = minimize(negative_log_marginal_likelihood, x0=x0,
652                   args=(X, y, gp.jitter, None, None, gp.kernel),
653                   method="L-BFGS-B", bounds=bounds)
654     if np.isfinite(res.fun) and res.fun < best_nll:
655         best_nll, best_theta = res.fun, res.x.copy()
656     ...
657 gp.theta_log = best_theta.copy()

```

HPO scheduling. `gp_cfg.hpo_every` (default 1, line 53) controls the period: `hpo_every=5` triggers HPO only every fifth iteration, substantially reducing overhead for cheap objectives.

5 Acquisition Functions

Acquisition functions quantify the utility of sampling at a candidate point $\mathbf{x}$ given the GP posterior (μ_*, σ_*) in the *original* (denormalised) scale. All three functions are returned by `make_acquisition()` (lines 766–838) as a vectorised callable $a : \mathbb{R}^{m \times d} \rightarrow \mathbb{R}^m$.

5.1 Expected Improvement (EI)

For *minimisation* with current best $y^\dagger = \min_i y_i$ and jitter $\xi \geq 0$:

$$\text{EI}(\mathbf{x}) = (y^\dagger - \mu_* - \xi) \Phi(Z) + \sigma_* \phi(Z), \quad Z = \frac{y^\dagger - \mu_* - \xi}{\sigma_*}, \quad (12)$$

where Φ and ϕ are the standard-normal CDF and PDF. EI is zero wherever $\sigma_* = 0$ (no uncertainty, no improvement possible).

5.2 Probability of Improvement (PI)

$$\text{PI}(\mathbf{x}) = \Phi\left(\frac{y^\dagger - \mu_* - \xi}{\sigma_*}\right). \quad (13)$$

PI ignores the *magnitude* of improvement and is more exploitative than EI.

5.3 Lower Confidence Bound (LCB/UCB)

For minimisation the acquisition is

$$\text{LCB}(\mathbf{x}) = \mu_* - \kappa \sigma_*, \quad (14)$$

which the internal optimiser maximises as $-\text{LCB} = -\mu_* + \kappa \sigma_*$. The parameter κ (line 73) trades exploration against exploitation.

Remark 3. Sign convention. The optimiser always *maximises* $a(\mathbf{x})$. For EI and PI this is natural; for LCB the code returns $-(\mu - \kappa \sigma)$ so that maximising it is equivalent to minimising the lower confidence bound.

6 Acquisition Optimisation

Finding $\mathbf{x}_{\text{next}} = \arg \max_{\mathbf{x} \in \mathcal{X}} a(\mathbf{x})$ is non-convex. Three strategies are available in `optimize_acquisition()` (lines 856–951), selected by `OptimConfig.method`.

Random (`method="random"`): draw N candidates uniformly from the bounding box (`n_raw_samples`, default 2000) and return `arg max`.

Grid (`method="grid"`): build a uniform Cartesian grid with `grid_n_per_dim` points per axis (default 20). Suitable only for $d \leq 3$ since the grid grows exponentially with dimension.

Refined (`method="refined"`): combine the global search above with local L-BFGS-B polishing. The top `n_restarts` (default 10) candidates seed `scipy.optimize.minimize` for precise local convergence. Recommended when the objective is expensive and precision matters.

Remark 4. Because a reuses the precomputed $(\alpha, \mathbf{L})$, each evaluation at m points costs $\mathcal{O}(mn)$. The random global scan costs $\mathcal{O}(Nn)$ total; the refined step adds $\mathcal{O}(KTn)$ where K is the number of restarts and T the average L-BFGS-B step count.

7 Rank-1 Cholesky Update

7.1 Motivation and Complexity

At each BO iteration the GP is refitted on a dataset growing by one point. The standard approach assembles the full $n \times n$ kernel matrix and calls `scipy.linalg.cholesky` at cost $\mathcal{O}(n^3)$. After T iterations with n_0 initial points, total fitting cost is $\mathcal{O}((n_0 + T)^4)$. The rank-1 update reduces each per-iteration cost to $\mathcal{O}(n^2)$, giving $\mathcal{O}((n_0 + T)^3)$ total — an asymptotic factor of $(n_0 + T)$ improvement.

7.2 Block Cholesky Extension

Given $\mathbf{K}_n^y = \mathbf{K}_n + \sigma_y^2 \mathbf{I}_n = \mathbf{L}_n \mathbf{L}_n^\top$, after adding $\mathbf{x}_{n+1}$ the augmented matrix is

$$\mathbf{K}_{n+1}^y = \begin{pmatrix} \mathbf{K}_n^y & \mathbf{k} \\ \mathbf{k}^\top & k_{n+1,n+1} + \sigma_y^2 + \delta \end{pmatrix}, \quad (15)$$

where $\mathbf{k} = [k(\mathbf{x}_i, \mathbf{x}_{n+1})]_{i=1}^n$ and δ is the jitter. The new Cholesky factor inherits the block structure

$$\mathbf{L}_{n+1} = \begin{pmatrix} \mathbf{L}_n & \mathbf{0} \\ \mathbf{l}^\top & l_{nn} \end{pmatrix}, \quad (16)$$

with the two new entries determined by matching $\mathbf{L}_{n+1} \mathbf{L}_{n+1}^\top = \mathbf{K}_{n+1}^y$:

$$\mathbf{l} = \mathbf{L}_n^{-1} \mathbf{k} \quad (\text{forward substitution, } \mathcal{O}(n^2)), \quad (17)$$

$$l_{nn} = \sqrt{k_{n+1,n+1} + \sigma_y^2 + \delta - \mathbf{l}^\top \mathbf{l}}. \quad (18)$$

Code. `chol_rank1_update()` (lines 420–465):

Listing 11: Cholesky extension in `chol_rank1_update()`

```

454 n = L.shape[0]
455 l = solve_triangular(L, k_cross, lower=True) # Eq. (7):  $\mathcal{O}(n^2)$ 
456 d = k_self + sigma_y**2 + jitter - float(np.dot(l, l))
457 l_nn = np.sqrt(max(d, jitter)) # Eq. (8): guard for
    r~0
458 L_new = np.zeros((n+1, n+1), dtype=float)
459 L_new[:n, :n] = L
460 L_new[n, :n] = l
461 L_new[n, n] = l_nn
462 return L_new

```

7.3 Alpha Update

After the Cholesky extension we need $\boldsymbol{\alpha}_{n+1} = \mathbf{K}_{n+1}^{y,-1} \tilde{\mathbf{y}}_{n+1}$, where all y values are in normalised space when `normalize_y=True`. Using the block structure of $\mathbf{L}_{n+1}$ the forward and backward sweeps give:

$$\mathbf{v}_1 = \mathbf{L}_n^\top \boldsymbol{\alpha}_n, \quad v_2 = \frac{\tilde{y}_{n+1} - \mathbf{l}^\top \mathbf{v}_1}{l_{nn}}, \quad \alpha_{n+1}^{(n+1)} = \frac{v_2}{l_{nn}}, \quad \boldsymbol{\alpha}_{n+1}^{(1:n)} = \mathbf{L}_n^{-\top} (\mathbf{v}_1 - \mathbf{l} \alpha_{n+1}^{(n+1)}). \quad (19)$$

The key insight is that $\mathbf{v}_1 = \mathbf{L}_n^{-1} \tilde{\mathbf{y}}_n = \mathbf{L}_n^\top \boldsymbol{\alpha}_n$ (since $\boldsymbol{\alpha}_n = \mathbf{L}_n^{-\top} \mathbf{L}_n^{-1} \tilde{\mathbf{y}}_n$), so $\mathbf{v}_1$ is recovered by a cheap $\mathcal{O}(n^2)$ mat-vec, without an extra triangular solve. Total cost: one mat-vec plus one backward solve, both $\mathcal{O}(n^2)$.

Code. `alpha_rank1_update()` (lines 468–512):

Listing 12: Alpha update in `alpha_rank1_update()`

```

497     l      = L_new[n, :n]
498     l_nn   = L_new[n, n]
499
500     #  $v_1 = L_{old}^T \alpha_{old}$  [Eq. (9),  $O(n^2)$  mat-vec, no extra solve]
501     v1 = L_old.T @ alpha_old
502
503     v2      = (y_new - float(np.dot(l, v1))) / l_nn
504     alpha_new_last = v2 / l_nn
505
506     rhs      = v1 - l * alpha_new_last
507     alpha_new_first = solve_triangular(L_old, rhs, lower=True, trans='T'
508 )
509
510     return np.concatenate([alpha_new_first, [alpha_new_last]])

```

7.4 Integration into the BO Loop

`update_gp_rank1()` (lines 515–561) assembles the cross-covariance $\mathbf{k}$ and uses $k_{n+1,n+1} = \sigma_f^2$ (valid for both kernels at $r = 0$). When normalisation is active the new observation is mapped to normalised space using the *existing* $(\bar{y}, s_y)$ before the update:

Listing 13: `update_gp_rank1()`

```

539     y_new_norm = (y_new_raw - gp.y_mean) / gp.y_std # existing stats:
540               no recompute
541
542     k_cross = kernel_amp(gp.Xs, x_new, gp.l_c, gp.sigma_f, kind=gp.
543               kernel).reshape(-1)
544     k_self   = float(gp.sigma_f**2) #  $k(x,x) = \sigma_f^2$  for both
545               kernels
546
547     L_new     = chol_rank1_update(gp.L, k_cross, k_self, gp.sigma_y, gp.
548               jitter)
549     alpha_new = alpha_rank1_update(gp.L, gp.alpha, L_new, y_new_norm)

```

In the main BO loop (lines 1144–1173) three fitting paths are selected:

Listing 14: Fitting path selection in `bayesian_optimization()`

```

1164     use_rank1 = (
1165         gp_cfg.rank1_threshold > 0
1166         and len(y) >= gp_cfg.rank1_threshold
1167         and gp.L is not None
1168     )
1169     if it == 0 or do_hpo:
1170         gp = fit_gp(gp, X, y, gp_cfg, it) # (a)  $O(n^3)$ ,
1171               renormalise
1172     elif use_rank1:
1173         gp = update_gp_rank1(gp, X[-1:], float(y[-1])) # (b)  $O(n^2)$ 
1174               rank-1
1175     else:
1176         gp = fit_gp(gp, X, y, gp_cfg, it) # (c) full, HPO
1177               skipped

```

- **Path (a)** is mandatory at iteration 0 and whenever HPO runs. New hyperparameters invalidate the entire kernel matrix, and the normalisation statistics are recomputed from scratch.
- **Path (b)** is triggered once the dataset has reached `rank1_threshold` points (`rank1_threshold > 0` and `len(y) >= rank1_threshold`) and no HPO occurs. At this point `X` and `y` already contain the new point while `gp.Xs` / `gp.ys` still hold the previous dataset, so `X[-1:]` and `y[-1]` identify the single new observation.
- **Path (c)** is the fallback full refit with HPO skipped.

Remark 5. When rank-1 updates are interleaved with periodic HPO (e.g. `hpo_every=5`), the pattern is: full refit at iterations 0, 5, 10, ... (with renormalisation and HPO), followed by four $\mathcal{O}(n^2)$ rank-1 steps using the frozen normalisation. The periodic full refit also resets any accumulated floating-point drift in the factors.

8 The BO Loop

The complete algorithm is in `bayesian_optimization()` (lines 1089–end). Algorithm 1 provides the pseudocode.

Algorithm 1 Bayesian Optimisation — `bayesian_optimization()`

Require: objective f , bounds $\mathcal{X}$, configs `bo_cfg`, `gp_cfg`, `acq_cfg`, `optim_cfg`

```

1: Sample  $n_0$  initial points (uniform or LHS, line 1128)
2: Build GP container  $\leftarrow$  build_gp_model() (line 1139)
3: for  $t = 0, 1, \dots, T - 1$  do
4:   if  $t = 0$  or HPO scheduled then
5:     Full refit incl. (re-)normalisation:  $(\alpha, L) \leftarrow \text{fit\_gp}()$  Path (a)
6:   else if rank1_threshold  $> 0$  and  $|\mathcal{D}| \geq \text{rank1\_threshold}$  then
7:      $(\alpha, L) \leftarrow \text{update\_gp\_rank1}()$  Path (b)
8:   else
9:     Full refit, HPO skipped Path (c)
10:  end if
11:   $y^\dagger \leftarrow \min_i y_i$  (raw scale)
12:   $a(\cdot) \leftarrow \text{make\_acquisition}(\text{acq\_cfg}, \text{gp}, y^\dagger)$  [GP in norm. space;  $a$  in raw scale]
13:   $\mathbf{x}_{t+1} \leftarrow \text{optimize\_acquisition}(a, \mathcal{X}, \text{optim\_cfg})$ 
14:   $y_{t+1} \leftarrow f(\mathbf{x}_{t+1})$ 
15:  Append  $(\mathbf{x}_{t+1}, y_{t+1})$  to  $(X, y)$ 
16: end for
17: return  $(\mathbf{x}^*, y^*) = \arg \min_{(x,y) \in \mathcal{D}} y$ 

```

9 Configuration Reference

All settings live in typed `dataclass` containers defined in lines 43–189 of `BO_ML4F.py`. Table 1 summarises the most important parameters.

Recommended configuration for a medium-cost problem.

```

gp_cfg = bo.GPConfig(
    kernel          = "matern52",  # twice-differentiable, more
    robust          = True,        # keeps sigma_f ~ 1 regardless
    normalize_y     = True,        # of y scale
    of_y_scale      = True

```

Table 1: Key configuration parameters.

Class	Field	Default	Description
GPConfig	l_c	0.3	Length-scale ℓ
	sigma_f	1.0	Kernel amplitude σ_f
	sigma_y	0.1	Noise std σ_y
	jitter	10^{-10}	Diagonal stabiliser δ
	kernel	"rbf"	"rbf" or "matern52" (§2)
	normalize_y	False	Centre/scale y before fitting (§3.3)
	optimize_hyperparams	False	Enable HPO
	hpo_every	1	Run HPO every k iterations
	n_hpo_restarts	3	Random restarts in HPO
	rank1_threshold	0	Dataset size ($\geq$) at which $\mathcal{O}(n^2)$ update engages; 0 disables it
AcqConfig	kind	"EI"	Acquisition type (EI/PI/UCB)
	xi	0.01	Exploration jitter ξ (EI/PI)
	kappa	2.0	Exploration weight κ (LCB)
	maximize	False	Set True for maximisation
OptimConfig	method	"random"	Acq. optimisation mode
	n_raw_samples	2000	Candidates for random/refined
	n_restarts	10	Local restarts in refined mode
BOConfig	n_init	5	Initial points
	n_iter	20	BO iterations
	init_sampling	"uniform"	Initial design ("lhs" available)

```

optimize_hyperparams = True,
hpo_every             = 5,                # full refit every 5 iterations
n_hpo_restarts       = 3,                # warm + default + 3 random = 5
    starts
rank1_threshold       = 500,              # O(n^2) update once dataset
    reaches 500 pts
)

```

10 Test Cases and Examples

The folder contains two test scripts exercising `BO_ML4F.py` directly: a 1D smoke-test and a 3D test. Earlier development also used a pair of legacy examples calling `BO_func.YL.py` directly and a scikit-optimize reference script; those were used only to validate the Matérn 5/2 kernel and output-normalisation features against a third-party implementation and are not part of this repository.

10.1 1D_Test_CASE.py — 1D example with BO_ML4F.py

This is the primary smoke-test for the new library. The objective is the one-dimensional noisy function

$$f(x) = \sin(5x)(1 - \tanh(x^2)) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 0.1^2), \quad x \in [-2, 2]. \quad (20)$$

The function has a clear global minimum near $x \approx 0$ flanked by shallower local minima; the tanh factor suppresses the oscillations away from the origin. The script first plots 100 dense evaluations for visual inspection, then launches `bayesian_optimization()` with all default settings:

Listing 15: 1D_Test_CASE.py — key call

```

res = bo.bayesian_optimization(
    f=func,

```

```

    bounds=[(-2.0, 2.0)],
    bo_cfg=bo.BOConfig(random_state=1234),
    gp_cfg=bo.GPConfig(),           # RBF kernel, no HPO, no rank-1
    acq_cfg=bo.AcqConfig(),         # EI, xi=0.01
    optim_cfg=bo.OptimConfig(),     # random search, N=2000
    save_cfg=bo.SaveConfig(
        plt_state_enabled=True,     # plot GP + EI at every iteration
        save_path="./out"
    ),
)

```

Because $d = 1$, `plt_state_enabled=True` is safe: the 1D plotting routine in `plt_state()` draws the true function, the GP posterior mean $\pm 2\sigma$, the observations, and the acquisition function at every BO iteration, saving each frame to `./out/`.

10.2 3D_Test_CASE.py — 3D example with BO_ML4F.py

This script tests the library on a three-dimensional objective:

$$f(\mathbf{x}) = \sin(3x_0) + \frac{1}{2} \cos(5x_1) + \frac{1}{5}x_2^2 + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 0.1^2), \quad \mathbf{x} \in [-2, 2]^3. \quad (21)$$

The script first generates 500 random points via `init_dataset()` and plots them in a 3D scatter coloured by $f(\mathbf{x})$, providing a visual sanity check of the objective's structure. It then runs the BO loop:

Listing 16: 3D_Test_CASE.py — key call

```

res = bo.bayesian_optimization(
    f=func,
    bounds=[(-2, 2), (-2, 2), (-2, 2)],
    bo_cfg=bo.BOConfig(
        n_init=10,
        n_iter=30,
        random_state=1234
    ),
    gp_cfg=bo.GPConfig(),
    acq_cfg=bo.AcqConfig(kind="EI"),
    optim_cfg=bo.OptimConfig(method="random"),
    save_cfg=bo.SaveConfig(
        plt_state_enabled=False,     # REQUIRED for d > 1
        save_path="./out_3d"
    ),
)

```

`plt_state_enabled` must be `False` for $d > 1$: the plotting routine only supports 1D inputs (it builds a dense grid on the single axis). The best solution is accessed via `res.best_x` and `res.best_y` after the loop.

10.3 Development-time references (not included in this repository)

Two kinds of throwaway scripts informed the design of `BO_ML4F.py` but were not kept in the repository: manual step-by-step ports of the BO loop built directly on `BO_func_YL.py` (used to trace every step of the algorithm without the `bayesian_optimization()` abstraction layer), and a scikit-optimize reference run (`skopt.gp_minimize` on the 1D objective of Eq. (20)) used to benchmark the Matérn 5/2 kernel (Section 2.2) and output normalisation (Section 3.3) against a third-party implementation before porting them into `BO_ML4F.py`. Neither is required to run or understand the current library.

11 Corrections to B0_func_YL.py

Two issues in the legacy module `B0_func_YL.py` were fixed in the July 2026 revision.

Vectorised EI. The original `propose_location()` called `EI()` in a Python `for` loop over all grid points. The revised version passes the entire grid matrix `grid_points` (shape $N_g \times d$) to a batch-capable `EI()` that calls the vectorised `gp.predict()`:

```
# Before (loop):
ei_values = np.array([EI(x, ...) [0] for x in grid_points])
# After (vectorised):
ei_values = EI(grid_points, X_train, y_train, params, xi, alpha, L)
```

The new `EI()` (lines 85–103 of `B0_func_YL.py`) accepts an (N, d) batch, returns an $(N,)$ array, and correctly zeros entries where $\sigma = 0$.

Stray tab character. A mixed-indent bug (tab + spaces) in `plot_n_save_B0()` caused an `IndentationError` under strict Python parsers; replaced with four spaces throughout.